Capstone 6 – AI Tutor Code (Google Colab)

This is the code for the AI tutoring and verification tool. Paste each cell into its own Colab cell and run them in order. The same code is also provided as `ai_coding_tutor_colab.ipynb` (upload to Colab) and `ai_coding_tutor_code.py`.

What it does

- Cell 1 runs any Python program and captures its real output.
- Cell 2 compares an AI's claimed output with the real output (CORRECT / WRONG / ERROR).
- Cell 3 scans code for outdated syntax and unsafe patterns.
- Cell 4 is the automated answer checker for practice problems (the automated step).
- Cell 5 grades the 10-question quiz.
- Cell 6 (optional) calls a free Gemini model and then verifies the answer. It needs a free API key in the `GEMINI_API_KEY` environment variable; check the current free model name in Google AI Studio and change `model` if needed.

No extra installs are needed for Cells 1–5.

Cell 1 – Run a program and capture real output

```
import subprocess, sys, tempfile, os, ast, re, json

def run_code(code, timeout=5):
    """Run Python source in a temp folder; return (stdout, stderr,
return_code)."""
    with tempfile.TemporaryDirectory() as folder:
        path = os.path.join(folder, "prog.py")
        with open(path, "w") as f:
            f.write(code)
        r = subprocess.run([sys.executable, path], capture_output=True,
                           text=True, timeout=timeout, cwd=folder)
    return r.stdout, r.stderr, r.returncode
```

Cell 2 – Verify what an AI claimed the output would be

```
def normalise(text):
    return "\n".join(line.rstrip() for line in text.strip().splitlines())

def verify_claim(code, claimed_output):
    out, err, rc = run_code(code)
    if rc != 0:
        return {"verdict": "ERROR", "actual": err.strip()}
    ok = normalise(out) == normalise(claimed_output)
    return {"verdict": "CORRECT" if ok else "WRONG", "actual": out.strip()}

# Example:
# code = open("p09_default_argument.py").read()
# print(verify_claim(code, "['apple']\n['pear']"))
```

Cell 3 – Risk scanner for outdated syntax and unsafe suggestions

```
def scan_risks(code):
    findings = []
    if re.search(r"^\s*print\s+[\"'\w]", code, re.M) and "print(" not in
code:
        findings.append("Python 2 print statement (outdated)")
    for old in ("raw_input", "xrange", "iteritems", "unicode("):
        if old in code:
            findings.append(f"Python 2 name '{old}' (outdated)")
    try:
        tree = ast.parse(code)
    except SyntaxError as e:
        return findings + [f"Syntax error: {e.msg} (line {e.lineno})"]
    in_with = {id(item.context_expr) for n in ast.walk(tree)
               if isinstance(n, ast.With) for item in n.items}
    for node in ast.walk(tree):
        if isinstance(node, ast.Call):
            name = getattr(node.func, "id", getattr(node.func, "attr", ""))
            if name in ("eval", "exec"):
                findings.append(f"Unsafe: {name}() runs arbitrary text as
code (line {node.lineno})")
            if name == "system":
                findings.append(f"Unsafe: os.system() can run shell commands
(line {node.lineno})")
            if name == "open" and id(node) not in in_with:
                findings.append(f"open() without 'with' - file may stay open
(line {node.lineno})")
```

```
        if isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef)):
            for d in node.args.defaults:
                if isinstance(d, (ast.List, ast.Dict, ast.Set)):
                    findings.append(f"Mutable default argument in
'{node.name}' (line {node.lineno})")
    return findings

# Example:
# print(scan_risks("x = eval(input('Number? '))"))
```

Cell 4 – Automated answer checker for practice problems

```
PRACTICE = {
    "sum_to": {"prompt": "Write sum_to(n) that returns 1+2+...+n.",
               "tests": [((1,), 1), ((5,), 15), ((10,), 55), ((0,), 0)]},
    "count_vowels": {"prompt": "Write count_vowels(text) that counts a, e, i,
o, u (any case).",
                     "tests": [(("hello",), 2), (("AEIOU",), 5),
(("rhythm",), 0), (("",), 0)]},
    "reverse_words": {"prompt": "Write reverse_words(s) that reverses the
ORDER of words.",
                      "tests": [(("a b c",), "c b a"), (("hello",), "hello"),
((""),), "")]},
}

def check_answer(problem, func):
    tests = PRACTICE[problem]["tests"]
    passed = 0
    for args, expected in tests:
        try:
            got = func(*args)
        except Exception as e:
            got = f"{type(e).__name__}: {e}"
        ok = got == expected
        passed += ok
        print(("PASS" if ok else "FAIL"), f"{problem}{args} -> {got!r}
(expected {expected!r})")
    print(f"Score: {passed}/{len(tests)}")
    return passed == len(tests)

# Student attempt (edit and re-run):
# def sum_to(n): return sum(range(1, n + 1))
# check_answer("sum_to", sum_to)
```

Cell 5 – Quiz auto-grader

```
QUIZ = [
 ("What numbers does range(1, 6) produce?", ["1 to 6", "1 to 5", "0 to 5", "2
to 6"], "B"),
 ("In p02, what is printed right after the number 1?", ["0", "Liftoff!",
"-1", "Nothing"], "B"),
 ("What does factorial(0) return in p03?", ["0", "1", "An error", "None"],
"B"),
 ("After fruits.insert(1, 'blueberry'), where is 'blueberry'?", ["Index 0",
"Index 1", "End of list", "Index 2"], "B"),
 ("What does fruits[-1] give?", ["First item", "Last item", "An error",
"Second item"], "B"),
 ("In p05, what is the value of squares?", ["[1, 9, 25]", "[4, 16, 36]", "[2,
4, 6]", "[1, 4, 9]"], "B"),
 ("What does counts.get(word, 0) do when word is new?", ["Raises KeyError",
"Returns 0", "Returns None", "Adds the word with 1"], "B"),
 ("Why use 'with open(...)' for files?", ["It is faster", "It closes the file
automatically", "It allows Python 2", "It encrypts the file"], "B"),
 ("In p09, what does the SECOND call print?", ["['pear']", "['apple',
'pear']", "['apple']", "An error"], "B"),
 ("What does text[::-1] do in p10?", ["Removes the last letter", "Reverses
the string", "Sorts the string", "Copies half"], "B"),
]

def grade_quiz(answers):
    """answers: list of 10 letters, e.g. ['B','A',...]"""
    score = 0
    for i, ((q, opts, right), given) in enumerate(zip(QUIZ, answers), 1):
        ok = given.upper() == right
        score += ok
        print(f"Q{i}: {'correct' if ok else 'wrong (answer ' + right + ')'}")
    print(f"Score: {score}/{len(QUIZ)}")
    return score
```

Cell 6 (optional) – Ask a free AI model, then verify it

```
def ask_ai(prompt, model="gemini-flash-latest"):
    key = os.environ.get("GEMINI_API_KEY")
    if not key:
        return "(no API key set - paste the prompt into a chat tool by hand
instead)"
    import google.generativeai as genai   # in Colab: !pip install -q google-
generativeai
    genai.configure(api_key=key)
    return genai.GenerativeModel(model).generate_content(prompt).text

def explain_and_verify(code):
    prompt = ("Explain this Python code line by line for a beginner, then
state the exact "
              "output it prints:\n\n" + code)
    print("RISKS:", scan_risks(code) or "none found")
    print("REAL OUTPUT:\n" + run_code(code)[0])
    print("AI EXPLANATION:\n" + ask_ai(prompt))

# Example:
# explain_and_verify(open("p09_default_argument.py").read())
```

How to run in Colab

1. Open colab.research.google.com → New notebook (or upload the .ipynb).
2. Upload the ten program files (zip provided) using the Files panel.
3. Run Cells 1–5 in order. Take a screenshot of each result for the test-results document.
4. For Cell 6, create a free key in Google AI Studio and set it with `os.environ["GEMINI_API_KEY"] = "your-key"` (never share or commit the key to GitHub).